NFPL Access Control Service: Design

1. The model in one paragraph

A **permission** is a verb engineering owns (`job.prepare`, `job.check`, `job.file`, `job.approve_sensitive`, … 18 in total, `GET /permissions`). A **role** is a named bundle of permissions that operations owns, stored as data and versioned. A **grant** gives one person one role over a **scope** (client, branch, direction, mode; any of them may be "all") for a **time window**. A **decision** is a pure function `decide(person, action, job, at_time)` that runs a fixed list of checks and returns allowed/denied with every check's result written out. Nothing that affects authority is ever overwritten: grants are revoked, role versions are superseded, users are deactivated, and every job event stores a snapshot of the job plus the grants that authorised it. Because `decide()` takes a time, history questions are answered by asking the same function about the past, not by a second code path.

```
users ──< grants >── roles ──< role_versions (permissions as of a date)
            │ scope: client? branch? direction? mode?  window: effective_from .. valid_to / revoked_at
jobs ──< job_events (snapshot + authority)     jobs ──< requests (cancel-filed / reopen, two people)
absences (who is away, optional delegate)      decision_log (every attempt to change something, incl. refusals)
```

The checks, in order (all are evaluated, so the explanation is complete): account active → account type may do this (staff / client / service) → job data complete → job is in a stage where this action makes sense → holds a permission for this action *whose grant scope covers this job* → separation-of-duties rules. Denied unless every check passes.

Stack: Python 3.11+, FastAPI, SQLite (WAL). One process, one file, no infrastructure, which matches 120 staff. Postgres would be a drop-in change for multiple app servers; the SQL is plain.

2. Key decisions and trade-offs

Scoped grants, not more roles. "Checker for ACME at Mumbai, imports only" is one grant, not a new role. This keeps the role list short (10 roles cover everyone in the sample) while scope does the narrowing. *Trade-off:* scope is limited to the four dimensions the brief gives. A future dimension (e.g. "value over ₹1 cr") would be a code change, a deliberate choice to keep `decide()` easy to reason about.

Grants only add. There are no "deny" grants. Restrictions that must always hold (maker ≠ checker, filed jobs are locked, client accounts can't hold staff roles) are rules in code, not data. *Trade-off:* less expressive, but "why can she?" is always answered by pointing at a grant, and nobody can accidentally create a deny that silently blocks a branch.

One policy, two compilations. `decide()` handles one job; `action_clause()` compiles the same rules into a SQL `WHERE` so lists and counts stay fast. A test runs `decide()` on every job for every user and asserts the SQL inbox matches exactly (Q13).

Evaluate on every request; no permission caches or tokens. At this size each decision is ~0.1 ms, so there is nothing to gain from caching and a lot to lose (stale access after a revocation).

Fail closed. Unknown user, unknown action, missing job data, an unexpected exception: all deny, with the reason written down (Q14).

Measured at scale (`python -m app.scale`, 500 users, 200,000 jobs, laptop): inbox page + exact count for a maker-everywhere with 35k waiting jobs ≈ 35 ms; narrow users < 1 ms; full job list page + count for an auditor ≈ 35 ms; one decision ≈ 0.1 ms; "who could change this job that day" across all 500 users ≈ 150 ms.

3. Answers to the questions

1. Should everyone see everything? No. You see a job if one of your grants with a view permission covers it, so visibility comes from the same grants as action. Every staff role includes `job.view`, so a Checker for ACME sees ACME jobs. Acting needs the *specific* permission (`job.check`) in scope *plus* the stage and separation rules. Seeing is broader than acting: an Auditor (Tarun) sees all 315 jobs and can change none. A job you may not see returns 404, not 403, so its existence isn't leaked. Client users get a redacted view (no `internal_*` fields, no staff names).

2. Maker everywhere, Checker for a few. Three grants: `Maker` for all clients/branches; `Checker` for client ACME; `Checker` for client BHARAT (Priya). The narrow one is one grant: `Checker` for client ACME, branch BOM, direction import, any mode (Rahul). The README lists everyone this way, generated from the data (`python -m app.cli access`), so the documentation can't drift from the truth.

3. Preparer is also qualified to approve. Refused. `check` and `file` carry a separation rule: the person in `prepared_by` may not do them, regardless of grants. The explanation names it precisely: Priya *holds* `job.check` for J000001 but fails `separation_of_duties`. Refusals are logged even though the transaction rolls back. Side

effect worth knowing: a one-person branch (Rohan at Kolkata) can prepare but never complete its own job. The system reports such a job as STUCK (`GET /jobs/{id}/eligible`) rather than quietly relaxing the rule.

4. Responsible person absent. Absence is recorded with dates and an optional delegate. Absence moves *work*, not *authority*. Jobs assigned to an absent person return to the pool for everyone eligible; the delegate sees them flagged "covering for Arjun" so they know they're first in line. The delegate does **not** inherit the absent person's permissions, so they only see jobs they could do themselves. When recording the absence, the response lists jobs the delegate can't cover, so that's never a surprise. Nobody can be assigned work while absent. Absence doesn't block an absent person's own login; that's an HR/SSO concern, noted in decisions as a flag.

5. Responsibilities change mid-job. Changes apply on the next request (no session caching). Finished work stands: if Gopal prepared a job yesterday and loses his grant today, the prepared checklist is still valid and the checker still has to be someone else. In-progress work is handed back: when a grant is revoked, expires, or a role loses a permission, every job assigned to that person that they can no longer move forward is unassigned, back to the pool, with an `unassigned` event explaining why. Planned changes (a transfer next month) are future-dated grants, so they start on time without anyone remembering to click.

6. Cancelling a filed job / reopening a closed one. Never directly. Filed and Cancelled jobs are locked: no edits, no direct cancel, not even by an admin. A person with `job.request_cancel_filed` / `job.request_reopen` in scope (branch Ops Managers) files a request with a written reason; a different person with `job.approve_sensitive` (Compliance) approves with a written note. The approver may be neither the requester nor the person who filed the job. Reopening clears `prepared_by` / `checked_by` / `filed_by`, so the job goes through the full maker → checker → filer cycle again under the same rules. This is precisely the incident from the brief: the filed job could not have been altered, and the attempt would be in the history.

7. Client's own employee. A separate account kind (`client`), permanently tied to one client in the database. Client accounts can only hold client-audience roles (only `job.view_client` exists), and any grant to them is forced to their own client. They can't act on anything, and they see a redacted view. The constraints are in code, so a careless admin can't misconfigure them.

8. Admin fixing a stuck job; admin self-escalation. The Admin role runs access, not customs work: it has no prepare/check/file. To unstick a job an admin uses `admin_fix`, which can only *release* the assignee, *reset* a Prepared/Checked job back to Confirmed, or clear a parser flag. It never moves a job forward, never touches Filed/Cancelled jobs, and always needs a written reason. Self-escalation is blocked four ways: (a) nobody can grant to themselves; (b) roles containing privileged permissions (`access.grant`, `access.role_manage`, `job.approve_sensitive`, `job.admin_fix`) need a second person to approve the grant, who may be neither the granter nor the grantee; (c) changing a role you hold, or making a role privileged, needs a second admin; (d) granting authority is itself scoped: Meera (Branch Access Admin, Mumbai) can only grant within Mumbai. The first two admins are created by a one-time bootstrap recorded in the audit as `system:bootstrap`.

9. The overnight parser. A `service` account with one permission, `job.sync`, which code forbids any human role from containing (and forbids service accounts from holding anything else). It upserts by `external_ref`: creates jobs as Confirmed, updates data only while still Confirmed, and is idempotent. It never changes stages. If the feed wants to change a job people are already working on, or move a job to a different client/branch/direction/mode, the change is **held**: the job gets a `needs_attention` note and a `parser_update_held` event with the proposed diff, and a human decides. Items it can't place (unknown client, bad mode, fields it may not set) are rejected into the audit trail, not guessed.

10. "Why can she do this and he can't?" `GET /explain/compare?users=sneha,rahul&action=check&job=J000001` runs `decide()` for both and prints only the checks that differ: *"Sneha Iyer can and Rahul Mehta cannot check J000001. The difference: permission_in_scope: Sneha → holds job.check via Checker for branch Mumbai (grant #7); Rahul → has job.check but not for this job (Checker grant #6: direction is import but the job's is export)."* Every denial reaching an API caller carries the same full explanation.

11. "Who could have changed this job on the 3rd?" `GET /jobs/J000003/who-could?date=2026-10-03`. The service collects every moment that day where anything relevant changed (job events, grants starting/ending/revoked, role versions, deactivations), rebuilds the job's state at each from its snapshots, and runs `decide()` as of each moment for every user. The answer lists each person with the actions and time windows they had and the grants that gave them, alongside what *actually* happened and every *refused* attempt. Example: before 11:00 J000003 was Filed, so only Neha (approve the reopen) could change it; from 11:00 Delhi makers could edit.

12. A new kind of role next quarter. `POST /roles` with a name and a set of existing permissions: no deploy. It's live immediately unless it contains a privileged permission, in which case a second admin approves it. Role edits create a new version (history keeps the old one) and reconcile every holder's queue. A genuinely new *verb* (a new permission or rule) is a code change, deliberately, since that's where safety lives.

13. “Jobs waiting for me” count matches what I can do. The inbox is defined as: *I can do the job’s next step (by `decide()`’s rules) AND it’s routed to me* (assigned to me, unassigned, or assigned to someone absent). It’s compiled to SQL from the same policy objects, with one sub-query per stage so each uses the `(stage, scope)` index. The count and the page come from the same predicate. `test_inbox_matches_what_each_person_can_do` checks every user against brute-force `decide()` over every job. Expired grants are reconciled before any list is served.

14. When the system is unsure. It says no, and says why: unknown user or action, missing job data, an exception (HTTP 500 with “nothing was changed”). Ambiguous automated input is held for a human (parser). Situations where the honest answer is “nobody can do this” (a job with no eligible checker) are surfaced as STUCK rather than resolved by loosening a rule.

4. Questions I added

What if two people act on the same job at once? Every job has a `version`; writes are conditional on it and callers can pass `expected_version`. The loser gets 409 “reload and try again” rather than silently overwriting.

Can moving a job change who can see it? Yes, so changing client/branch/direction/mode requires `job.edit` for both the old and the new scope, plus a reason. A Delhi maker can’t push a job into Mumbai.

Does a person who left still have access? Users are deactivated, never deleted. Karan’s history on the 3rd is still answerable, and he can do nothing today.

Should denials be recorded? Yes. “Nobody could say who had been allowed to” was the incident; refused attempts are often the more interesting evidence. They’re in `decision_log`, shown in job history and in who-could.

5. Assumptions and what I cut

Assumptions. Stages: Confirmed → Prepared → Checked → Filed, plus Cancelled and Reopened (a Reopened job is re-prepared). The checker can send a checklist back. “Closed” means Filed or Cancelled. The filer must not be the preparer (the checker may file). Ownership routes work but isn’t required to act: any eligible person can claim from the pool, and a job assigned to a present colleague must be claimed via a manager reassign. Times are UTC.

Cut, and why. *Authentication:* identity is an `X-User` header standing in for the company SSO; the service is the authorisation layer, and wiring SSO is integration, not design. *A UI:* the API (with Swagger at `/docs`) plus a CLI covers the demo. *Notifications* (email the delegate, alert on parser holds): the data to drive them exists (`needs_attention`, events). *Field-level permissions* beyond the client redaction. *Multi-server deployment:* SQLite is enough for 120 staff; swapping to Postgres needs no model change. *Bulk grant import* (e.g. from HR): the API takes one grant at a time.